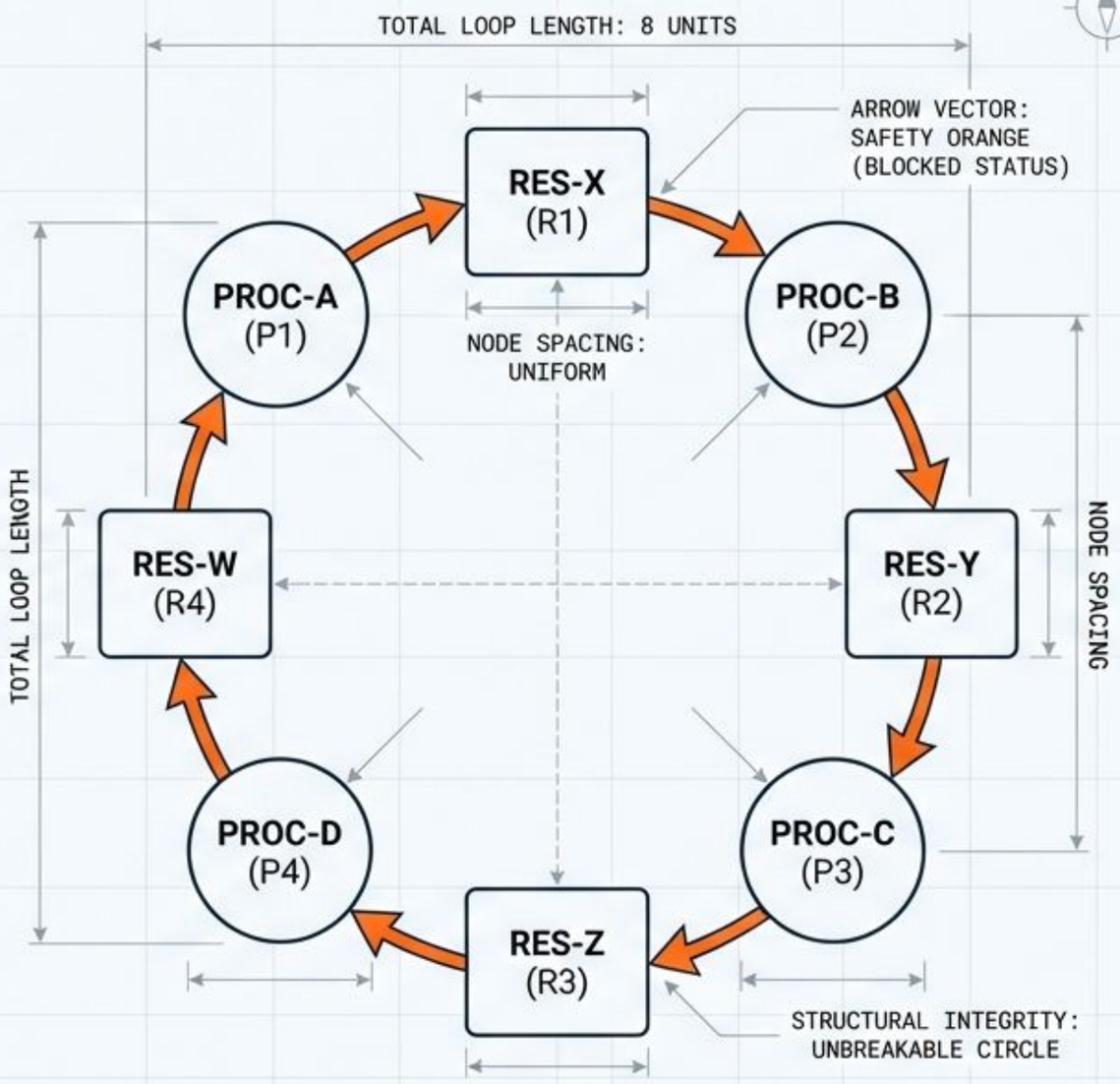
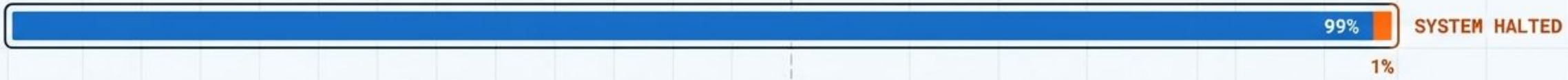
Process Deadlocks

System Architecture & Handling

- **Unit 3 Core Architecture:** Operating System Concept Mastery
- Anatomy of a System Failure (The Deadlock Phenomenon)
- Visual Diagnostics via Resource-Allocation Graphs (RAG)
- The Four Handling Pillars: Ignore, Prevent, Avoid, Detect





Prepared By : Er. Sharat Maharjan

The Deadlock Phenomenon

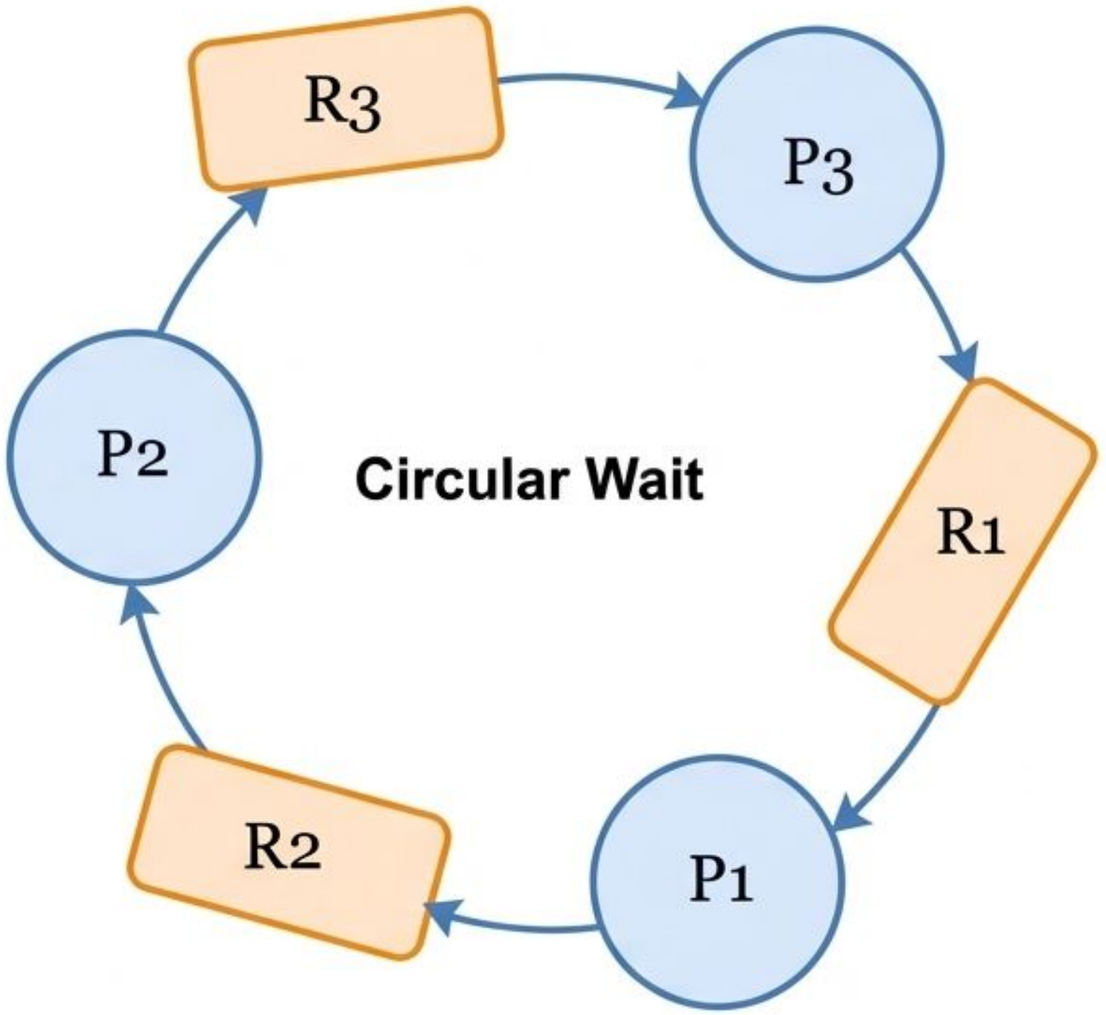
A permanent blocking condition where a set of processes remain indefinitely waiting because each process holds resources needed by another process in the set.

1. The Trigger: Each process holds resources while simultaneously requesting resources held by others.

2. The Result: A closed-loop circular wait where absolutely zero processes can proceed.

3. Required Elements: ≥ 2 processes, holding ≥ 1 resource, actively requesting additional resources.

⚠️ **Real-World Analogy:** A 4-way traffic intersection where four cars block each other completely.



P1 is waiting for P2 to release R2 , P2 is waiting for P3 to release R3 and P3 is waiting for P1 to release R1

Diagnostic Anatomy: The Coffman Conditions

Mutual Exclusion

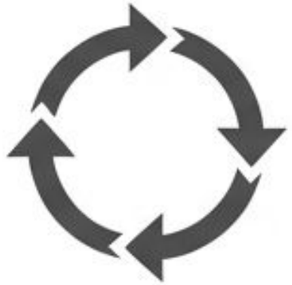
Resources are strictly non-sharable (e.g., printers, write-locks).

Hold and Wait

Processes retain current resources while actively waiting for new ones.

No Preemption

Resources cannot be forcibly stripped; they must be voluntarily released.



Circular Wait

A closed topological chain of processes exists, each waiting on the next.

EXAM KEY: ALL FOUR conditions must execute simultaneously to trigger a system deadlock.

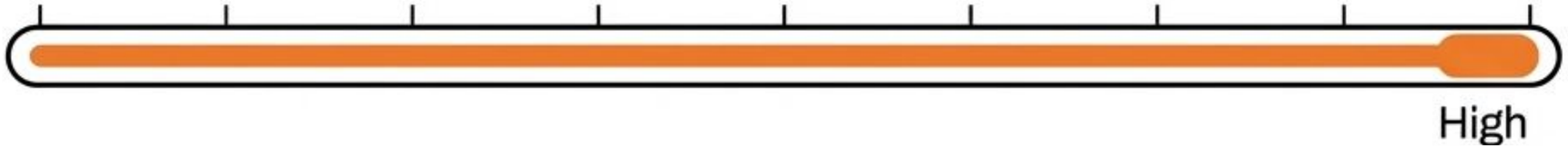
RESOLUTION KEY: Breaking just **ONE** of these conditions prevents the deadlock entirely.

Resource Vulnerability Matrix

	Preemptable Resources	Non-Preemptable Resources
Definition	Can be safely stripped from a process without causing execution failure.	Forcible removal instantly causes critical process failure or data corruption.
Examples	• CPU cycles • Main memory • Network bandwidth	• Printers • Tape drives • Database record locks
System Impact	Highly flexible, enables fair sharing, rarely causes deadlocks.	The primary culprit behind system deadlocks; low allocation flexibility.

Allocation Flexibility

Deadlock Risk

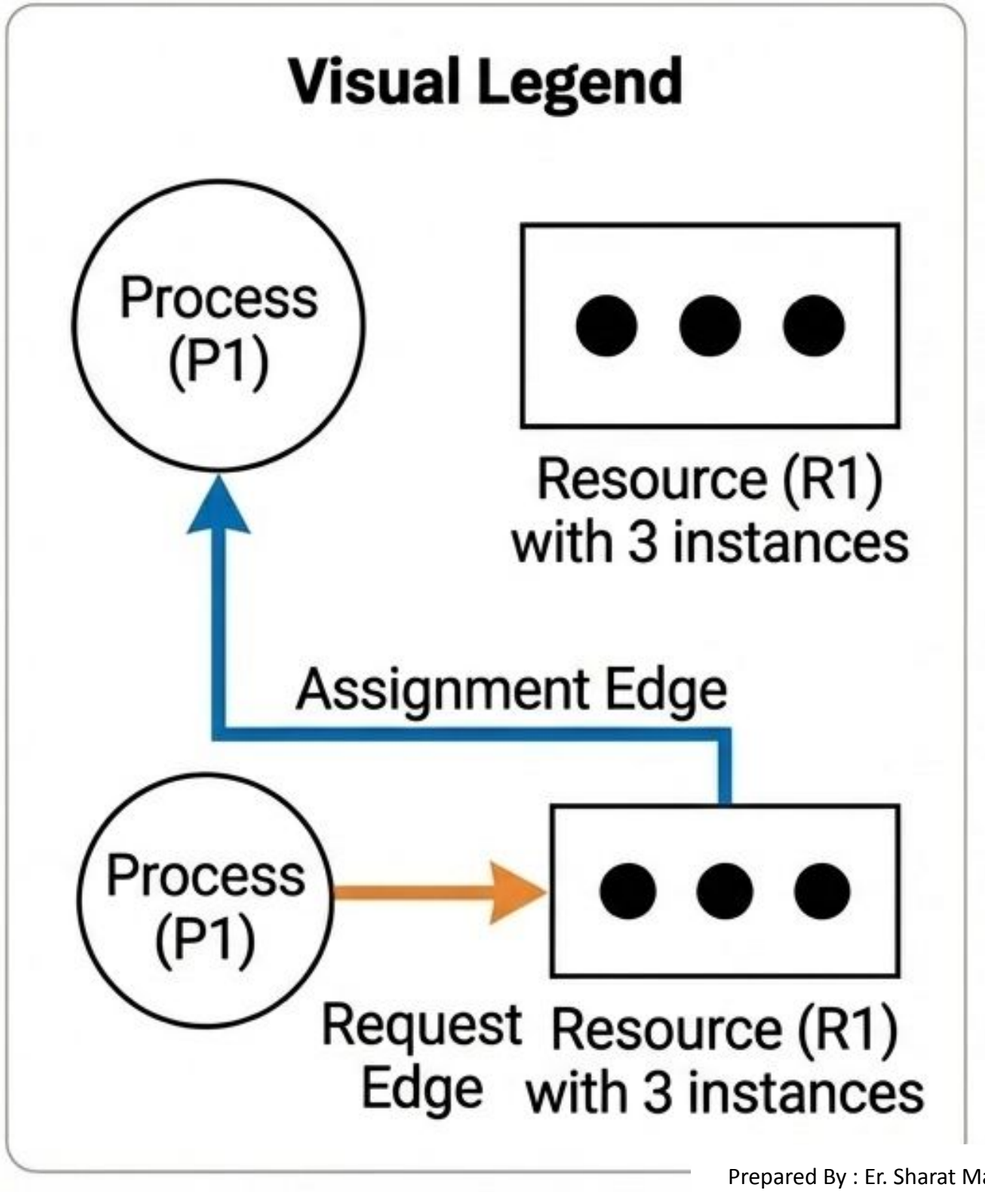


Modeling System State: Resource-Allocation Graphs (RAG)

Purpose: Directed mathematical graphs mapping current resource assignments and pending requests.

- **Process Nodes:** Represented as Circles.
- **Resource Nodes:** Represented as Rectangles (interior dots = number of available instances).
- **Assignment Edge (R → P)**: Resource instance is currently locked by a process.
- **Request Edge (P → R)**: Process is actively blocked, requesting a resource.

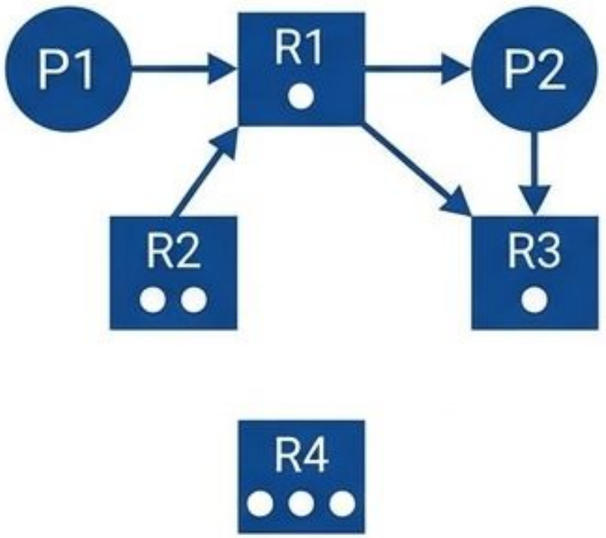
Diagnostic Rule:
No cycles = 100% Deadlock-Free.
Cycles = Potential Deadlock (requires further analysis).



Decoding RAG Topologies

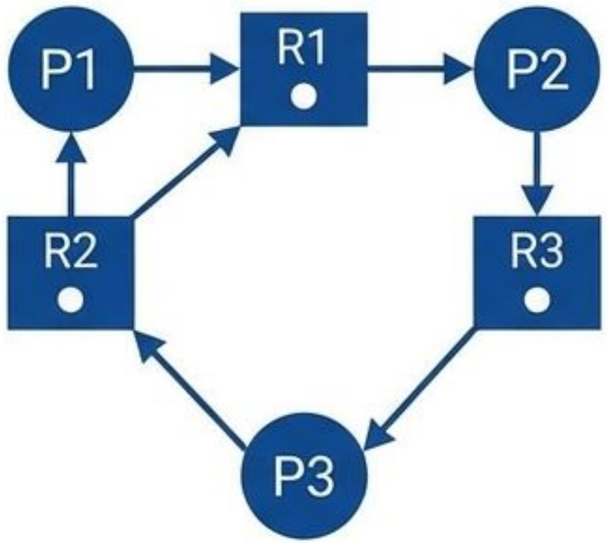
STATUS: SAFE STATE

No graph cycles exist. System operations flow normally without blocking.



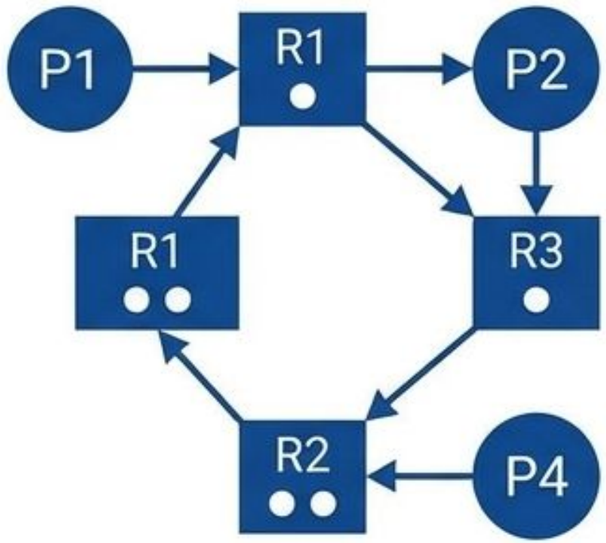
STATUS: TRUE DEADLOCK

Cycle exists utilizing single-instance resources. Absolute system deadlock.



STATUS: CYCLE BUT SAFE

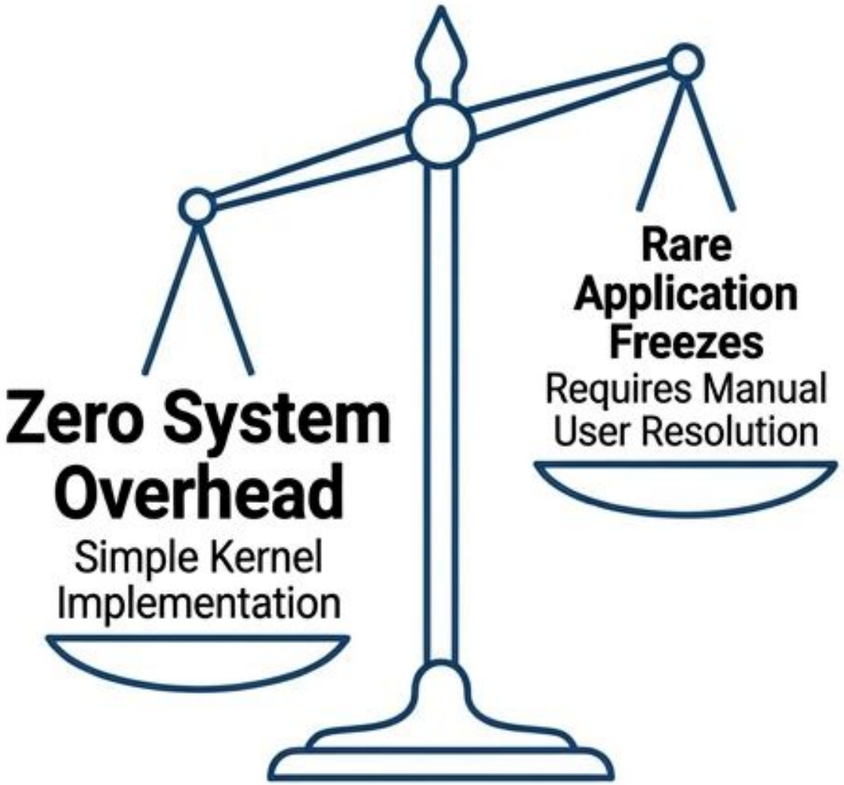
Cycle exists, but multiple-instance resources provide an escape route. Process P4 can release an instance of R2 to break the chain.



Strategy 1: The Ostrich Algorithm

Core Philosophy:
Deliberately ignore deadlocks; mathematically pretend they do not happen.

Target Use Case:
General-purpose Operating Systems (Unix, Linux, Windows desktop).



The Justification:
Deadlocks are statistically rare; the CPU overhead of prevention drastically exceeds the impact of occasional failure.

Critical Disadvantages:
Unacceptable for mission-critical systems (aerospace, medical hardware). Relies entirely on manual intervention.

Strategy 2: Absolute Prevention

Goal: Architect the kernel to strictly enforce the failure of ≥ 1 Coffman condition.
Trade-off: Guarantees safety but severely restricts resource utilization.

Force Hardware Sharing

Implement read-only files and printer spooling so resources are never strictly locked by one process.

Total Upfront Allocation

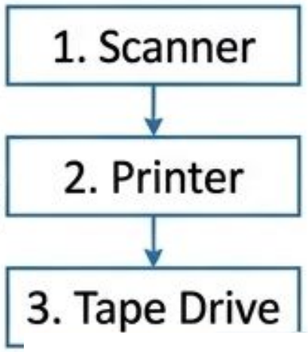
Require processes to request all resources at startup, or only when holding none.

Forcible Preemption

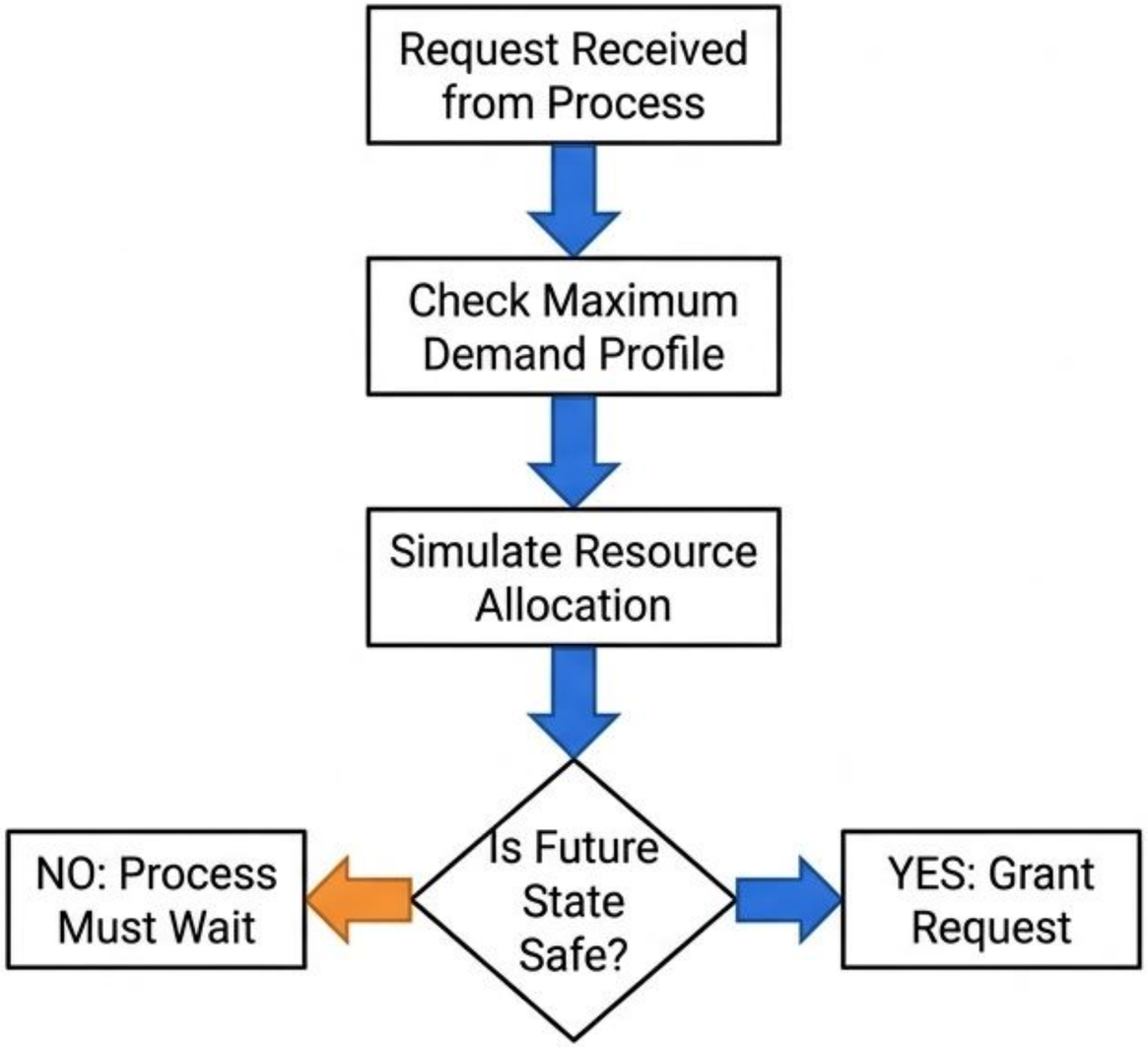
Implement preemptable CPU scheduling and virtual memory page reclaiming to strip resources when needed.

Strict Numerical Ordering

Impose a total ordering hierarchy. A process must request resources in ascending numerical order only.



Strategy 3: Dynamic Avoidance



The Prerequisite: The kernel requires advance knowledge of a process's Maximum Demand.

Core Formula:

$$\text{Need} = \text{Max Demand} - \text{Currently Allocated}$$

Safe State Definition: The system can logically allocate resources to all processes to completion without triggering a deadlock. Analyzed using Dijkstra's Banker's Algorithm.

Banker's Algorithm: Validating a Safe Sequence

Event: Process P3 requests 1 resource. Available = 3.

Step 1 & 2: Simulate

Process	Currently Has	Max Demand	Remaining Need
P1	2	6	4
P2	1	5	4
P3	3 (was 2)	5	2 (was 3)
P4	2	6	4

New Available Pool = 2.
P3 Request Valid.

Step 3 & 4: Safety Scan

P3 executes. Returns resources. Work = 5 → P1 executes. Returns resources. Work = 7 → P2 executes. Returns resources. Work = 8 → P4 executes. Returns resources. Work = 10

CONCLUSION: Safe Sequence Found: < P3 → P1 → P2 → P4 >. Request **GRANTED.**

Banker's Algorithm: Blocking a Deadlock

Event: Process P4 requests 1 resource. Initial Available = 3.

Step 1 & 2: Simulate

Process	Remaining Need
P1	4
P2	4
P3	3
P4	3 (was 4)

New Available Pool = 2.

Step 3: Safety Scan

```
Current Available (Work) = 2
Check P1: Need 4 > 2 (FAIL)
Check P2: Need 4 > 2 (FAIL)
Check P3: Need 3 > 2 (FAIL)
Check P4: Need 3 > 2 (FAIL)
```

CONCLUSION: ALL current Needs > Available (2). Zero processes can finish. System is deadlocked. Request **DENIED**.

Strategy 4: Continuous Detection

Core Philosophy: Allow deadlocks to occur naturally, detect them via background scanning, and forcibly recover the system.

The Frequency Dilemma: High frequency checks = crippling CPU overhead.
Low frequency checks = system remains paralyzed for longer durations.

Wait-For Graph Simplification

Single-Instance Resources: System periodically scans simplified Wait-For graphs for closed loops.

Multiple-Instance Resources: Uses a modified Banker's Algorithm to scan for permanently unfinished processes.

Emergency Resolution: Recovery Protocols

Deadlock Detected:
OS Triggers Emergency Resolution

Path A: Process Termination

Brute Force Resolution

- **Abort All:** Guaranteed instant fix, but results in massive data and computation loss.
- **Abort One-by-One:** Minimal data loss, but requires running cycle detection repeatedly until fixed.

Path B: Resource Preemption

Surgical Resolution

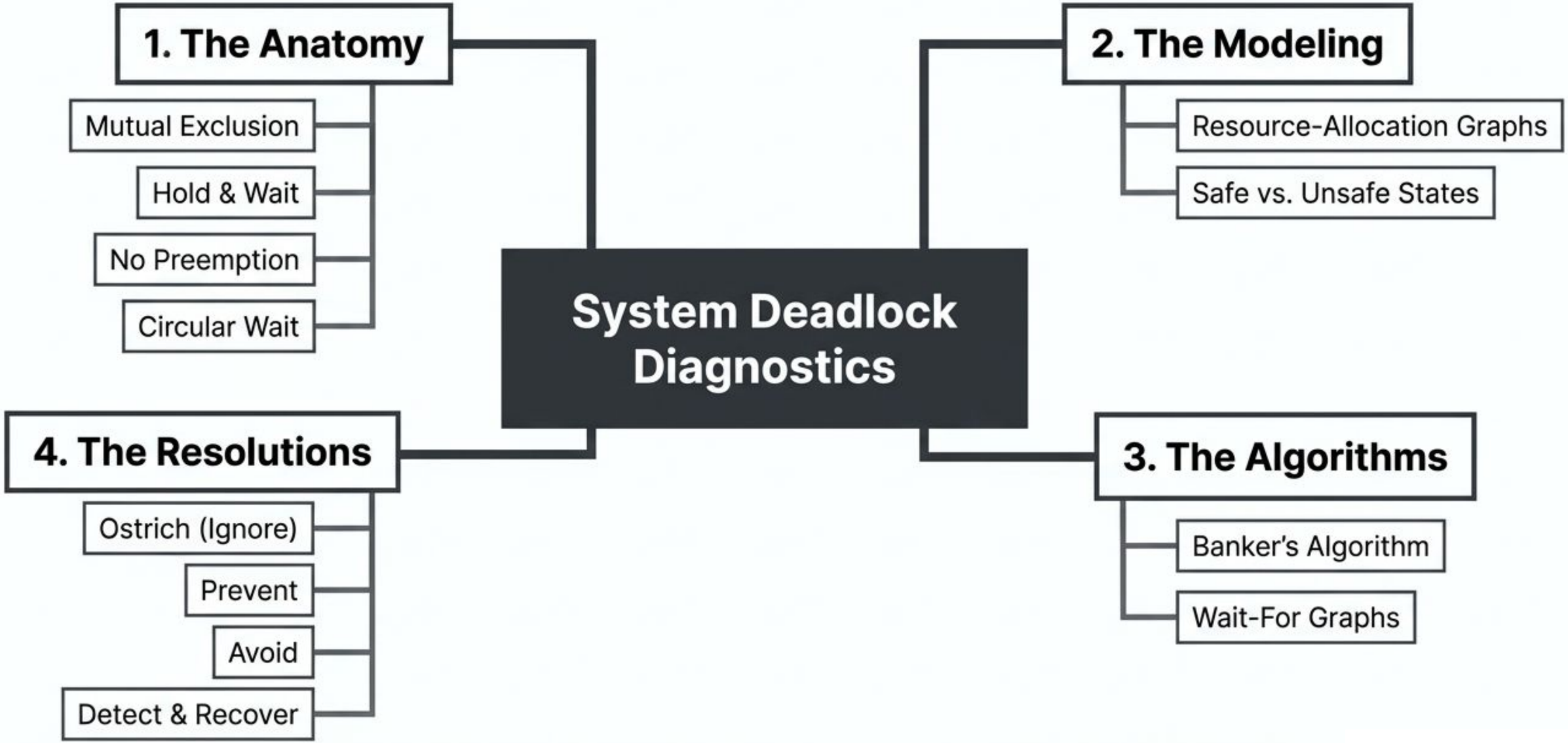
1. Select Victim (Minimize cost metric)
2. Rollback System State (Revert to safe checkpoint)
3. Reallocate Resources
4. Restart Victim Later

Challenge: Resource Preemption carries a high risk of Starvation if the exact same victim is picked repeatedly.

Strategic Synthesis: OS Handling Paradigms

Strategy	Principle	Key Advantage	Critical Disadvantage	Target Use Case
Ostrich (Ignore)	Deliberately pretend deadlocks do not happen.	Zero system overhead.	Highly unreliable.	General-purpose OS
Prevention	Architecturally break Coffman conditions.	Guarantees absolute safety.	Severe resource underutilization.	Mission-critical systems
Avoidance	Runtime checks for Safe States.	Balanced resource use.	Requires advance max demand info.	Medium-criticality systems
Detection	Periodic background cycle scanning.	High system flexibility.	Requires complex recovery downtime.	Fault-tolerant systems

Core Architectural Insight: Modern operating systems do not rely on one monolithic method. They mix these four strategies dynamically depending on the specific hardware and software resource types being requested.

Unit 3 Architecture: The Complete Picture

Master these interconnected pillars to mathematically diagnose and resolve any system deadlock scenario.